

Lecture 10

Integrated Machine-Learning Case Studies, Numerical Reliability, and Course Review

Instructor: Dr. M. Asad Iqbal, Assistant Professor
Department of Information Technology
Semester: 4th Semester, BS IT

Lecture Focus

Combining the course methods into reliable machine-learning workflows for regression, recommendation, dimensionality reduction, and practical model evaluation. [\[Data Eng. Link: Model Deployment, Data Pipelines\]](#)

1 Lecture Overview

Machine-learning systems rarely use one linear-algebra method in isolation. A realistic workflow represents data as matrices, transforms features, fits a model, evaluates errors, and checks whether the computation is stable and meaningful.

This final lecture integrates the course through applied case studies and a reproducible Python mini-project. The emphasis is not only obtaining an answer, but selecting an appropriate method and defending the result.

Learning Outcomes

After completing this lecture, students should be able to:

- Translate an applied problem into vectors, matrices, and dimensions.
- Construct regression, recommendation, and PCA workflows.
- Connect least squares, regularization, SVD, and embeddings.
- Select a suitable numerical solver for a matrix problem.
- Diagnose ill-conditioning, rank deficiency, and data leakage.
- Evaluate residual, prediction, and reconstruction errors.
- Build and interpret an end-to-end NumPy mini-project.
- Communicate mathematical choices, assumptions, and limitations.

A Reusable Applied Workflow

Stage	Central question
Represent	What do rows, columns, entries, and shapes mean?
Prepare	Must features be centered, scaled, encoded, or split?
Compute	Which factorization, solver, or transformation is appropriate?
Validate	Does the result generalize beyond the fitted data?
Diagnose	Are rank, conditioning, residuals, or leakage problematic?
Communicate	Can another person reproduce and interpret the result?

2 Case Study 1: Regression and Regularization

Suppose $X \in \mathbb{R}^{n \times d}$ contains n observations and d features, while $\mathbf{y} \in \mathbb{R}^n$ contains a continuous target. With an intercept column included, the linear model is

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon}.$$

Least squares chooses

$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \|X\boldsymbol{\beta} - \mathbf{y}\|_2^2.$$

The fitted values and residuals are

$$\hat{\mathbf{y}} = X\hat{\boldsymbol{\beta}}, \quad \mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}.$$

2.1 Ridge Regularization

When predictors are strongly correlated or the problem is poorly conditioned, ridge regression solves

$$\hat{\boldsymbol{\beta}}_{\lambda} = \arg \min_{\boldsymbol{\beta}} (\|X\boldsymbol{\beta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_2^2), \quad \lambda \geq 0.$$

Its normal-equation form is

$$(X^T X + \lambda I) \hat{\boldsymbol{\beta}}_{\lambda} = X^T \mathbf{y}.$$

The intercept is usually excluded from the penalty.

2.2 Evaluation

On an unseen test set of size m :

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}.$$

Compare the model against a simple baseline, such as predicting the training-target mean.

Practical Decisions

Split the data before estimating means, standard deviations, or model parameters. Fit every preprocessing step using the training set only, then apply the learned transformation to the test set.

[ML/DS Connection: Least squares provides the modeling objective, projections explain the fitted vector, and ridge regularization stabilizes weak or correlated feature directions.]

3 Case Study 2: Recommendation Systems

Let $R \in \mathbb{R}^{m \times n}$ be a user-item interaction matrix. Rows represent users, columns represent items, and observed entries may contain ratings, purchases, clicks, or other preference signals.

3.1 Similarity-Based Recommendation

Two users represented by row vectors $\mathbf{r}_i$ and $\mathbf{r}_j$ can be compared using cosine similarity:

$$s_{ij} = \frac{\mathbf{r}_i^T \mathbf{r}_j}{\|\mathbf{r}_i\|_2 \|\mathbf{r}_j\|_2}.$$

Recommendations can be generated from items preferred by similar users. Sparse overlap and popularity effects must be handled carefully.

3.2 Latent-Factor Recommendation

A rank- k approximation is

$$R \approx U_k \Sigma_k V_k^T.$$

Define

$$P = U_k \Sigma_k^{1/2}, \quad Q = V_k \Sigma_k^{1/2}.$$

Then

$$R \approx PQ^T, \quad \hat{r}_{ij} = \mathbf{p}_i^T \mathbf{q}_j.$$

The rows of P are user embeddings and the rows of Q are item embeddings. Their inner product estimates compatibility.

3.3 Real-World Caution

Missing interactions are not necessarily negative preferences. Ordinary SVD on a matrix filled with zeros may distort the meaning of unobserved entries. Production methods often minimize error only over observed interactions and may include bias and regularization terms.

Ranking Evaluation

Prediction error alone may not match the business objective. Recommendation systems are commonly evaluated with ranking measures such as precision at k , recall at k , coverage, and performance for new or less-active users.

[ML/DS Connection: Recommendation systems connect vector similarity, low-rank approximation, embeddings, and dot-product scoring in one commercially important application.]

4 Case Study 3: PCA in a Learning Pipeline

Let $X \in \mathbb{R}^{n \times d}$ contain correlated numerical features. Center the training data and compute

$$X_c = U\Sigma V^T.$$

The first k principal directions are the columns of V_k , and the reduced representation is

$$Z_k = X_c V_k \in \mathbb{R}^{n \times k}.$$

4.1 Selecting the Number of Components

The cumulative explained-variance ratio is

$$\text{CEV}(k) = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}.$$

A threshold such as 90% or 95% can guide exploration, but the best k should also be checked using validation performance, interpretability, memory, and computation.

4.2 Reconstruction and Compression

Reconstruct the original feature space using

$$\hat{X} = Z_k V_k^T + \mathbf{1}\mu^T.$$

The relative reconstruction error is

$$\frac{\|X - \hat{X}\|_F}{\|X\|_F}.$$

4.3 Correct Pipeline Order

1. Split observations into training and test sets.
2. Estimate centering and scaling values from the training set.
3. Fit PCA on the transformed training matrix.
4. Transform both sets using the training principal directions.
5. Fit the downstream model on training scores.
6. Evaluate on test scores without refitting preprocessing.

[ML/DS Connection: PCA maximizes retained input variance, not target prediction. A component with modest variance can still be useful for classification or regression, so downstream validation remains necessary. PCA can reduce storage and training time, remove correlated directions, support visualization, and sometimes improve numerical conditioning.]

5 Numerical Reliability and Solver Choice

Mathematically equivalent formulas can behave differently in floating-point computation. Reliable machine learning requires attention to rank, scale, and conditioning.

5.1 Conditioning

For a full-rank matrix,

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

A large condition number means that small changes in data may cause large changes in a solution. Forming normal equations can make the problem worse because

$$\kappa_2(A^T A) = \kappa_2(A)^2.$$

5.2 Choosing a Computation

Problem	Preferred tool
Square $Ax = b$	<code>np.linalg.solve</code>
Least squares	<code>np.linalg.lstsq</code>
Rank-deficient system	SVD or pseudoinverse
Symmetric positive definite	Cholesky factorization
Dimensionality reduction	Truncated SVD or PCA
Repeated large problems	Iterative methods

5.3 Reliability Checklist

- Scale features when units differ substantially.
- Inspect rank, singular values, and condition number.
- Avoid explicit matrix inversion when solving a system.
- Use tolerances rather than exact equality for floating-point results.
- Compare alternative solvers on small test cases.
- Report sensitivity, uncertainty, and known data limitations.

[ML/DS Connection: Ridge regression increases weak eigenvalues of $X^T X$ by λ . This reduces sensitivity but introduces bias. The value of λ should be selected using validation data.]

6 Validation, Diagnostics, and Communication

A result is useful only when it is mathematically correct, numerically reliable, and evaluated for the intended task.

6.1 Core Diagnostics

Diagnostic	Question answered
System residual $\ Ax - b\ _2$	Does the solution satisfy the equations?
Relative residual	Is the residual small relative to the data?
Prediction RMSE	How large are unseen prediction errors?
PCA reconstruction error	How much matrix information was discarded?
Condition number	Is the computation sensitive?
Baseline comparison	Does the model add practical value?

6.2 Frequent Failure Modes

- **Data leakage:** test information influences scaling, PCA, or fitting.
- **Shape mismatch:** rows and columns represent inconsistent objects.
- **Silent rank deficiency:** redundant features make parameters unstable.
- **Metric mismatch:** the reported measure does not reflect the application goal.
- **Over-interpretation:** correlation or vector proximity is treated as causation.

Minimum Reproducible Report

State the problem, data-matrix meaning, dimensions, preprocessing, method, parameter choices, evaluation split, numerical diagnostics, results, and limitations. Include executable code with a fixed random seed.

[ML/DS Connection: Technical work is market-ready when another analyst can reproduce it, verify the assumptions, and explain why the selected method supports the decision being made.]

7 Capstone: Applied Linear Algebra Portfolio Project

Select one dataset and solve a meaningful problem using at least two methods from the course. The project should demonstrate mathematical understanding, Python implementation, and professional communication.

7.1 Suggested Project Tracks

- **Regression:** compare least squares and ridge under correlated features.
- **Recommendation:** compare cosine neighbors with a low-rank latent-factor model.
- **Compression:** compare PCA or truncated SVD across several ranks.
- **Network data:** study an adjacency or graph-feature matrix using eigenvectors, similarity, or low-rank representations.

7.2 Required Deliverables

1. A one-paragraph problem statement and clearly defined objective.
2. A description of rows, columns, units, dimensions, and missing values.
3. Mathematical formulation of the selected methods.
4. Reproducible Python code using NumPy and relevant libraries.
5. At least one baseline and one appropriate evaluation measure.
6. Shape, rank, condition, or reconstruction diagnostics where relevant.
7. Two clear visualizations or tables and a concise interpretation.
8. A README explaining how to run the project and reproduce the results.

7.3 Suggested Assessment Rubric

Criterion	Marks
Problem formulation and data representation	15
Mathematical correctness and method selection	25
Python implementation and reproducibility	20
Validation and numerical diagnostics	20
Interpretation, limitations, and communication	20
Total	100

[ML/DS Connection: A strong project does not need the largest dataset. A smaller, well-explained analysis with correct mathematics and honest validation is more valuable than complicated code without interpretation.]

8 Python Mini-Project: Reliable Ridge Regression

The following example uses a built-in numerical dataset, keeps test data separate, standardizes features using training statistics, solves a ridge system, and compares test RMSE with a baseline.

```

1 import numpy as np
2 from sklearn.datasets import load_diabetes
3 from sklearn.model_selection import train_test_split
4
5 X, y = load_diabetes(return_X_y=True)
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.25, random_state=7
8 )
9
10 # Learn preprocessing from the training set only.
11 mean = X_train.mean(axis=0)
12 std = X_train.std(axis=0)
13 Xtr = (X_train - mean) / std

```

```

14 Xte = (X_test - mean) / std
15
16 # Add an intercept column.
17 A = np.column_stack([np.ones(len(Xtr)), Xtr])
18 A_test = np.column_stack([np.ones(len(Xte)), Xte])
19
20 # Ridge system; do not penalize the intercept.
21 lam = 1.0
22 penalty = np.eye(A.shape[1])
23 penalty[0, 0] = 0.0
24 beta = np.linalg.solve(
25     A.T @ A + lam * penalty,
26     A.T @ y_train
27 )
28
29 prediction = A_test @ beta
30 rms = np.sqrt(np.mean((prediction - y_test) ** 2))
31
32 baseline = np.full_like(y_test, y_train.mean())
33 baseline_rms = np.sqrt(np.mean((baseline - y_test) ** 2))
34
35 r2 = 1 - np.sum((prediction - y_test) ** 2) / np.sum(
36     (y_test - y_test.mean()) ** 2
37 )
38
39 print("Train/test shapes:", A.shape, A_test.shape)
40 print("Condition number:", np.linalg.cond(A))
41 print("Model RMSE:", rms)
42 print("Baseline RMSE:", baseline_rms)
43 print("Test R-squared:", r2)

```

Interpret the Output

The model should be judged on unseen test observations and against the baseline. The condition number describes sensitivity, while RMSE and R^2 describe predictive performance. This educational example is not a clinical decision system.

Extend the experiment by comparing several values of λ selected on a validation set, and compare the result with `np.linalg.lstsq`.

9 Course Review and Next Steps

No.	Core idea	Machine-learning connection
01	Linear systems	Constraints, data equations, solution structure
02	Vectors and span	Feature representation and combinations
03	Basis and dimension	Redundancy and compact coordinates
04	Linear transformations	Matrix mappings and model layers
05	Orthogonal projections	Similarity, decomposition, approximation
06	Least squares	Regression and residual minimization
07	Eigenpairs	Spectral directions and repeated dynamics
08	SVD and PCA	Low-rank models, reduction, compression
09	Networks, embeddings, attention	Modern learned representations
10	Integrated workflows	Reliable application and communication

9.1 Final Mastery Checklist

You are ready to apply the course when you can:

- Identify the meaning and dimensions of every matrix in a problem.
- Connect geometric interpretation with algebraic computation.
- Choose a stable solver instead of relying on an inverse formula.
- Implement and verify a method using NumPy.
- Compare results with a baseline and an appropriate error measure.
- Explain assumptions, limitations, and practical consequences clearly.

Practice Tasks

Choose one formula from each lecture and explain: its dimensions, geometric meaning, computational method, and one machine-learning application. This creates a compact revision portfolio for the complete course.

[ML/DS Connection: Course completion: You have progressed from vectors and systems to SVD, PCA, neural networks, attention, and reliable applied workflows.]